

3 The BuildTree Algorithm

Training Data Reminder

We are given a labelled dataset of n examples:

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n, \quad \mathbf{x}^{(i)} \in \mathbb{R}^d, \quad y^{(i)} \in \{0, 1, \dots, K\}.$$

Each example $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_d^{(i)})$ is a d -dimensional feature vector; $y^{(i)}$ is its class label. BUILDTREE is called with the full index set $I = \{1, \dots, n\}$ and partitions these examples recursively. At each node it holds a *subset* of indices $I \subseteq \{1, \dots, n\}$; the root starts with all n examples and each split divides I into two disjoint subsets passed to the child nodes.

3.1 The Classification BuildTree Algorithm

BuildTree for Classification

The algorithm takes a set of training indices I and a minimum leaf size k , and returns a tree node. Call BUILDTREE($\{1, \dots, n\}; k$) to build the full tree.

Algorithm 1 BUILDTREE(I, k)

```

1: if  $|I| \leq k$  or all labels in  $I$  are identical then
2:   return Leaf( $\hat{y}$  = majority class in  $I$ )           ▷ base case: node is pure or too small
3: end if
4:  $\text{Gini}^* \leftarrow \infty$ 
5: for each feature  $j \in \{1, \dots, d\}$  and candidate threshold  $s$  do
6:    $I^+ \leftarrow \{i \in I : x_j^{(i)} \geq s\}, \quad I^- \leftarrow \{i \in I : x_j^{(i)} < s\}$ 
7:    $\text{Gini}(j, s) \leftarrow \frac{|I^+|}{|I|} G(I^+) + \frac{|I^-|}{|I|} G(I^-)$ 
8:   if  $\text{Gini}(j, s) < \text{Gini}^*$  then
9:      $(j^*, s^*) \leftarrow (j, s); \quad \text{Gini}^* \leftarrow \text{Gini}(j, s)$ 
10:  end if
11: end for
12:  $I^+ \leftarrow \{i \in I : x_{j^*}^{(i)} \geq s^*\}, \quad I^- \leftarrow \{i \in I : x_{j^*}^{(i)} < s^*\}$ 
13: return Node( $j^*, s^*, \text{left} = \text{BUILDTREE}(I^-, k), \text{right} = \text{BUILDTREE}(I^+, k)$ )

```

3.2 Complexity of BuildTree

Per-Step Complexity of BuildTree(I, k)

The same algorithm is reproduced below with the cost of each step annotated in **orange**. Throughout, $m = |I|$ denotes the number of examples at the current node, d the number of features, and n the total training set size.

```

1: if  $|I| \leq k$  or all labels in  $I$  are identical then           ▷  $O(m)$  — one scan to check purity
2:   return Leaf( $\hat{y}$  = majority class in  $I$ )                     ▷  $O(m)$  — one pass to tally class counts
3: end if
4:  $\text{Gini}^* \leftarrow \infty$                                        ▷  $O(1)$ 
5: for each feature  $j \in \{1, \dots, d\}$  and candidate threshold  $s$  do   ▷  $d$  features;  $O(m)$ 
   thresholds each  $\Rightarrow O(dm)$  iterations
6:    $I^+ \leftarrow \{i \in I : x_j^{(i)} \geq s\}, \quad I^- \leftarrow \{i \in I : x_j^{(i)} < s\}$   ▷  $O(1)$  per threshold using sorted
   order + running counts
7:    $\text{Gini}(j, s) \leftarrow \frac{|I^+|}{|I|} G(I^+) + \frac{|I^-|}{|I|} G(I^-)$   ▷  $O(1)$  — update running class-count sums
8:   if  $\text{Gini}(j, s) < \text{Gini}^*$  then
9:      $(j^*, s^*) \leftarrow (j, s); \quad \text{Gini}^* \leftarrow \text{Gini}(j, s)$            ▷  $O(1)$ 
10:  end if
11: end for           ▷ sorting each feature costs  $O(m \log m)$ ; loop total:  $O(dm \log m)$ 
12:  $I^+ \leftarrow \{i \in I : x_{j^*}^{(i)} \geq s^*\}, \quad I^- \leftarrow \{i \in I : x_{j^*}^{(i)} < s^*\}$   ▷  $O(m)$  — one scan to partition
13: return Node( $j^*, s^*, \text{left} = \text{BUILDTREE}(I^-, k), \text{right} = \text{BUILDTREE}(I^+, k)$ )  ▷  $O(1)$  to
   form node; recurse on disjoint halves

```

Summary.

- **Per node call on m examples:** $O(dm \log m)$ — dominated by sorting each feature's values to enumerate thresholds efficiently.
- **Full tree (balanced):** Each of the $O(\log n)$ levels partitions all n examples across its nodes, so work per level is $O(dn \log n)$. Total: $\mathbf{O(dn \log^2 n)}$.
- **Full tree (worst case, depth n):** $O(dn^2)$ — e.g. when one leaf gains only one example per split.

4 Full Worked Example: Run Chase

Second-Innings Run Chase — Running BUILDTree for Classification

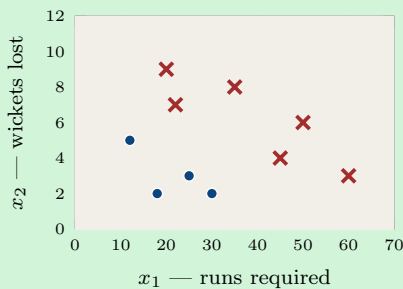
Setting: 10 historical match snapshots taken mid-innings. We want to classify whether the batting team *won* ($y = 1$) or *lost* ($y = 0$).

Features: x_1 = runs still required, x_2 = wickets lost so far.

Match	x_1 (runs req.)	x_2 (wkts lost)	y (result)
1	18	2	Win (1)
2	25	3	Win (1)
3	12	5	Win (1)
4	30	2	Win (1)
5	22	7	Lose (0)
6	20	9	Lose (0)
7	45	4	Lose (0)
8	50	6	Lose (0)
9	60	3	Lose (0)
10	35	8	Lose (0)

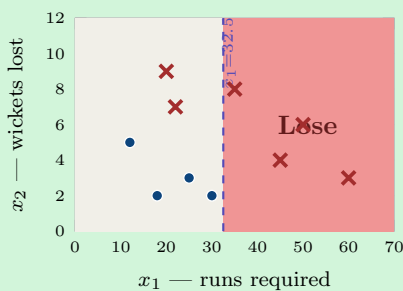
Figure 1 shows how BUILDTree carves up the feature space step by step.

Step 0 — full feature space



Before any split. All 10 match snapshots are plotted in the (x_1, x_2) plane. The entire feature space is *undecided* (grey). The root node holds all $n = 10$ examples with 4 wins ($\bullet$) and 6 losses ($\times$). Root Gini impurity:

$$G(S) = 2 \cdot \frac{4}{10} \cdot \frac{6}{10} = 0.480.$$

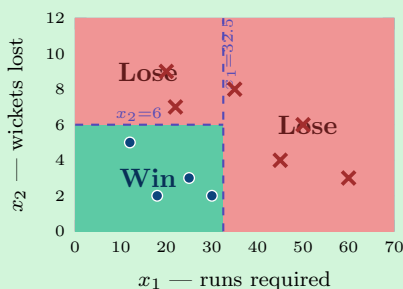
Step 1 — split $x_1 < 32.5$ 

Best root split: $x_1 < 32.5$. This vertical cut ties with $x_2 < 5.5$ at weighted Gini = 0.267; we break the tie by feature index.

Left ($x_1 < 32.5$): 4W/2L, $G = 0.444$

Right ($x_1 \geq 32.5$): 0W/4L, $G = 0.000$

The right child is *immediately pure* — it becomes a **Lose** leaf. The left child is still mixed and must be split further.

Step 2 — split $x_2 < 6$ 

Best split within $x_1 < 32.5$: $x_2 < 6$. This horizontal cut achieves weighted Gini = 0, perfect purity in both children.

$x_2 < 6$ (matches 1–4): 4W/0L → **Win** leaf

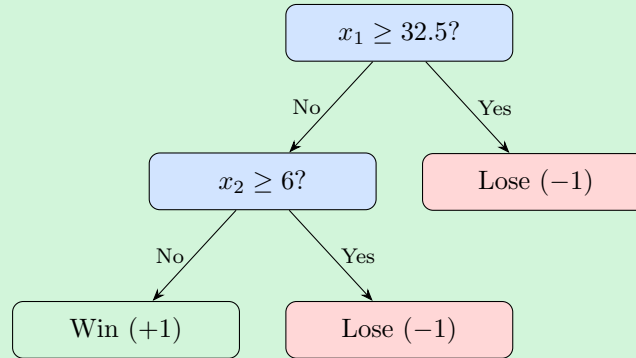
$x_2 \geq 6$ (matches 5–6): 0W/2L → **Lose** leaf

All three leaves are now pure. BUILDTree stops. The tree uses only 2 of the available features and achieves **100% training accuracy**.

■ Win region
 ■ Lose region
 Undecided
 ● Win (match) ✕ Lose (match)

Figure 1: Step-by-step partition of the (x_1, x_2) feature space by BUILDTREE. Each row adds one axis-aligned split (dashed purple line). Filled circles (●) are winning matches; crosses (✕) are losses.

Final tree (100% training accuracy):



Reading off the rules:

1. If runs required $\geq 32.5 \rightarrow$ **Lose** (target too large to chase).
2. If runs required < 32.5 and wickets lost $< 6 \rightarrow$ **Win** (reachable target with wickets in hand).
3. If runs required < 32.5 and wickets lost $\geq 6 \rightarrow$ **Lose** (target manageable but too few wickets remaining).

The tree has discovered an **interaction**: wickets lost only become decisive when the runs target is small enough to be chaseable. This joint condition cannot be represented by a logistic regression without manually engineering a cross-feature term.

Classifying a new match snapshot. A team needs 28 more runs ($x_1 = 28$) and has lost 4 wickets ($x_2 = 4$). Walk the tree:

1. **Root:** Is $x_1 \geq 32.5$? $28 \geq 32.5$ — **No** $\rightarrow$ go left.
2. **Node:** Is $x_2 \geq 6$? $4 \geq 6$ — **No** $\rightarrow$ go left.
3. **Leaf:** **Win** (+1).

The target is small enough to be chaseable and the team still has enough wickets — the tree predicts a win.

5 Extended Worked Example: Deeper Run Chase Tree

Run Chase — A Deeper Tree with an Unexpected Rule

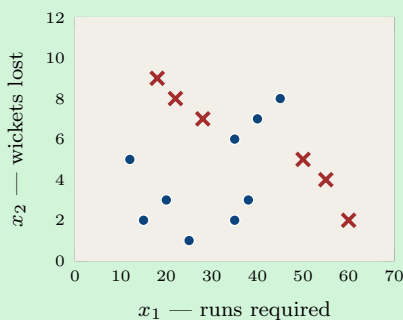
Setting: 15 historical match snapshots. Same prediction task ($y = 1$ win, $y = 0$ lose) and same two features as before, but with more data and a more varied pattern — forcing the tree to grow deeper.

Features: x_1 = runs still required, x_2 = wickets lost.

Match	x_1 (runs req.)	x_2 (wkts lost)	y (result)
1	15	2	Win (1)
2	20	3	Win (1)
3	12	5	Win (1)
4	25	1	Win (1)
5	22	8	Lose (0)
6	18	9	Lose (0)
7	28	7	Lose (0)
8	35	6	Win (1)
9	40	7	Win (1)
10	45	8	Win (1)
11	35	2	Win (1)
12	38	3	Win (1)
13	50	5	Lose (0)
14	55	4	Lose (0)
15	60	2	Lose (0)

Figure 2 traces how BUILD TREE partitions the (x_1, x_2) feature space across four splits. The third split (Step 3) re-uses x_1 at a *different* threshold, producing a nested vertical cut restricted to one sub-region of the plane.

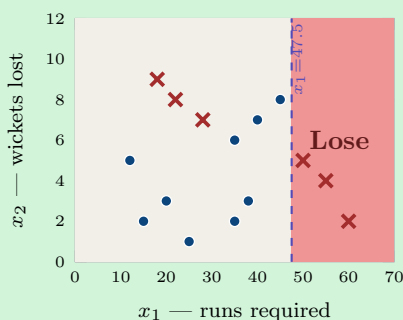
Step 0 — full feature space



Before any split. All 15 snapshots plotted. 9 wins ($\bullet$), 6 losses ($\times$). Root Gini:

$$G(S) = 2 \cdot \frac{9}{15} \cdot \frac{6}{15} = 0.480.$$

BUILD TREE searches both features and all thresholds. The best candidate is $x_1 < 47.5$.

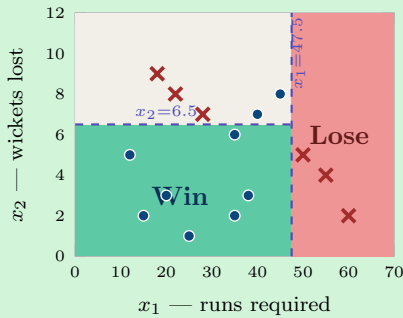
Step 1 — split $x_1 < 47.5$ 

Best root split: $x_1 < 47.5$. Weighted Gini = 0.300.

Left ($x_1 < 47.5$): 9W/3L, $G = 0.375$

Right ($x_1 \geq 47.5$): 0W/3L, $G = 0.000$

The right child is pure: **Lose** leaf (matches 13–15 — big totals, impossible to chase). The left child (mixed) is expanded next.

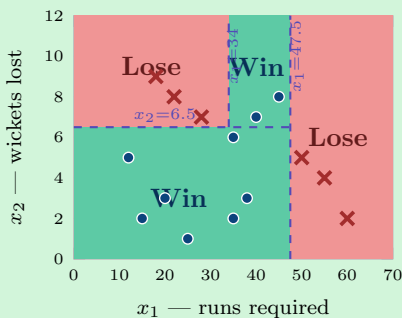
Step 2 — split $x_2 < 6.5$ (within $x_1 < 47.5$)

Within $x_1 < 47.5$: best split $x_2 < 6.5$.
Weighted Gini = 0.200.

$x_2 < 6.5$ (matches 1–4, 11, 12): 7W/0L → **Win**

$x_2 \geq 6.5$ (matches 5–10): 2W/3L, mixed

The lower region is a pure **Win** leaf. The upper band ($x_2 \geq 6.5$, $x_1 < 47.5$) still holds 2 wins and 3 losses and needs another split.

Step 3 — split $x_1 < 34$ (within $x_2 \geq 6.5$)

Within $x_2 \geq 6.5$, $x_1 < 47.5$: best split $x_1 < 34$. Weighted Gini = 0.000.

$x_1 < 34$ (matches 5–7): 0W/3L → **Lose**

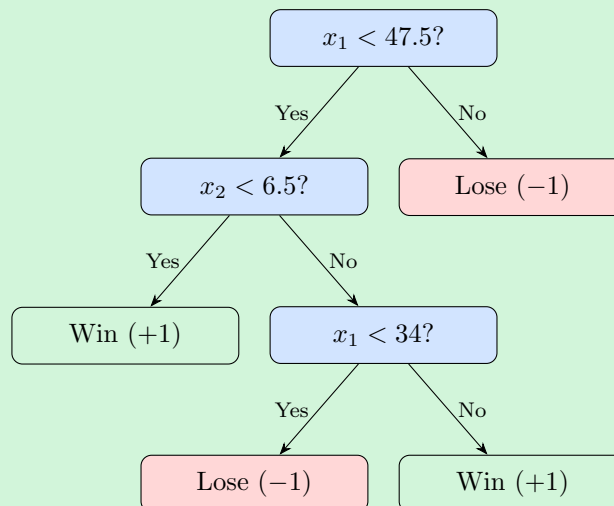
$x_1 \geq 34$ (matches 9, 10): 2W/0L → **Win**

Surprising rule: matches 9 and 10 have many wickets lost ($x_2 = 7, 8$) yet the tree predicts **Win**. The deciding factor is that their runs required ($x_1 = 40, 45$) sit in the window $[34, 47.5)$ — a middling chase that is large enough to suggest the batting side has been scoring freely, implying they still have the momentum to close out the game.

■ Win region ■ Lose region ■ Undecided ● Win (match) ✕ Lose (match)

Figure 2: Step-by-step partition of the (x_1, x_2) plane for the extended example. The third split (Step 3) re-uses x_1 at threshold 34, creating a *nested* vertical cut within the high-wicket band ($x_2 \geq 6.5$).

Final tree (100% training accuracy):



Reading off the rules:

1. If $x_1 \geq 47.5 \rightarrow$ **Lose** (total too large to chase).
2. If $x_1 < 47.5$ and $x_2 < 6.5 \rightarrow$ **Win** (manageable chase with wickets in hand).

3. If $x_1 < 47.5$, $x_2 \geq 6.5$, and $x_1 < 34 \rightarrow$ **Lose** (many wickets lost, chase too tight to recover).
4. If $x_1 < 47.5$, $x_2 \geq 6.5$, and $x_1 \geq 34 \rightarrow$ **Win** (many wickets lost, but enough runs and time to absorb the pressure).

What makes this tree more interesting than the previous example:

- x_1 appears at *two* levels of the tree with different thresholds (47.5 at the root, 34 deeper). This is a non-linear, non-convex use of a single feature that no linear classifier can replicate.
- Rule 4 is counter-intuitive: a team with many wickets lost ($x_2 \geq 6.5$) still wins — but only when runs required falls in $[34, 47.5)$. This middling chase window is what the tree has discovered: big enough that the batting side has been scoring freely, yet small enough to finish.
- The tree has 4 internal nodes, 5 leaves, and uses only 2 features — a compact, fully interpretable model.

Classifying a new match snapshot. A team needs 36 more runs ($x_1 = 36$) and has lost 7 wickets ($x_2 = 7$). Walk the tree:

1. **Root:** Is $x_1 < 47.5$? $36 < 47.5$ — **Yes** $\rightarrow$ go left.
2. **Node:** Is $x_2 < 6.5$? $7 < 6.5$ — **No** $\rightarrow$ go right.
3. **Node:** Is $x_1 < 34$? $36 < 34$ — **No** $\rightarrow$ go right.
4. **Leaf:** **Win** (+1).

Despite 7 wickets lost, the runs required sits in the window $[34, 47.5)$ — the tree predicts a win.